InferenceLens

Inference Cost / Quality Tradeoff Auditor

Interview Defense Document · Sidharth Kriplani · 2025

Answers the question: **How does an AI system know it's working correctly when cost and quality are competing constraints?** InferenceLens makes the cost/quality Pareto frontier explicit, measurable, and defensible — with empirically-grounded routing rules and a full audit trail.

1. The Problem Being Solved

AI systems make implicit cost/quality tradeoffs on every inference call. Most engineering teams route all calls to the largest available model by default, or apply hand-tuned heuristics with no systematic validation. Neither approach produces an auditable paper trail of *why* a routing decision was made.

The failure mode: **AI inference routing with no auditability.** When gpt-4o handles a sentiment classification that gpt-3.5-turbo would solve at 94% quality for 97% less cost — that is a routing failure. InferenceLens finds it, proves it, and generates defensible IF/THEN routing rules backed by empirical evidence.

Failure Mode	What Goes Wrong	InferenceLens Fix
Over-routing to large model	97% cost premium on tasks small model handles well	Pareto frontier identifies dominated configs
Under-routing to small model	Quality below acceptable floor for high-stakes tasks	Quality floor enforcement per complexity band
No routing audit trail	Cannot explain why a model was chosen	Append-only JSONL audit per inference call
Silent quality regression	Routing rules degrade as model pricing changes	Re-profiling triggers new Pareto analysis

2. Component Architecture

Component	File	Role
InferenceProfiler	pipeline/profiler.py	Runs tasks across model tiers; logs tokens, latency, cost per call
QualityEvaluator	pipeline/evaluator.py	ROUGE-1 (summarization), macro F1 (classification), exact match (extraction
CostTracker	pipeline/cost_tracker.py	USD cost per call; aggregates by tier and task type
ParetoAnalyzer	pipeline/pareto.py	Identifies Pareto frontier; flags dominated configs
RoutingRecommender	pipeline/router.py	Generates IF/THEN routing rules per complexity band
AuditLogger	pipeline/audit.py	Append-only JSONL audit of all inference and routing decisions
ReportGenerator	pipeline/report.py	Assembles InferenceLensReport: Pareto data, routing rules, PASS/WARN/FA

3. Pareto Frontier Analysis

A model configuration is **dominated** if there exists another config that achieves strictly higher quality at strictly lower cost. A config on the **Pareto frontier** cannot be improved on one dimension without sacrificing the other.

Dominance criterion:

```
Config A dominates config B if: cost(A) <= cost(B) AND quality(A) >= quality(B), with at least one strict inequality.
```

Worked example — summarization task:

Tier	Model	Avg Cost / call	Quality Score	Pareto Status
large	gpt-4o	$0.0042	0.91	FRONTIER
medium	gpt-4o-mini	$0.00013	0.88	FRONTIER
small	gpt-3.5-turbo	$0.00025	0.79	DOMINATED by medium
local	ollama/mistral	$0.00	0.74	FRONTIER

gpt-3.5-turbo is dominated by gpt-4o-mini: medium achieves higher quality (0.88 vs 0.79) at lower cost ($0.00013 vs $0.00025). The small tier is dropped from routing consideration.

4. Routing Recommender — IF/THEN Rules

The RoutingRecommender converts Pareto analysis into executable routing rules stratified by **complexity score** (0–1, derived from prompt token count, vocabulary entropy, and task type).

Generated routing rules for summarization:

```
IF complexity < 0.35 (simple): route to LOCAL (ollama/mistral) — $0.00/call, quality 0.74
```

```
IF 0.35 <= complexity < 0.70 (standard): route to MEDIUM (gpt-4o-mini) — $0.00013/call, quality 0.88
```

```
IF complexity >= 0.70 (complex): route to LARGE (gpt-4o) — $0.0042/call, quality 0.91
```

Rules are enforced with a **quality floor** of 0.75 and a **cost savings minimum** of 20% — a model tier is only recommended if it saves at least 20% vs. the next tier up. These thresholds are configurable in `config.py`.

5. Quality Evaluation Methodology

Quality metrics are task-type-specific to reflect what actually matters in production — not a generic similarity score applied uniformly.

Task Type	Primary Metric	Secondary Metric	Why
Summarization	ROUGE-1 (0.6 weight)	Semantic similarity (0.4)	Content overlap matters; paraphrase acceptable
Classification	Macro F1 (0.7 weight)	Semantic similarity (0.3)	Class correctness is non-negotiable

Extraction	Exact match (0.5 weight)	Semantic similarity (0.5)	Structured fields require precision

Semantic similarity uses sentence-transformers cosine similarity as a fallback when reference outputs are unavailable or for tasks where paraphrase is acceptable. This prevents hard failures when exact-match metrics would incorrectly penalise correct but differently-worded outputs.

6. Expected Interview Questions & Answers

Q: Why build this as an auditor rather than just a router?

A: A router makes decisions. An auditor proves them. The goal is not just to route calls correctly — it is to produce a defensible record showing why each call was routed the way it was. In production, when an engineering or finance team asks 'why are we spending this much on inference?', the audit log answers the question with empirical evidence rather than assumptions.

Q: How does Pareto dominance handle the case where one model is better on all metrics?

A: If model A dominates all other models (lower cost AND higher quality), the Pareto frontier contains only model A, and routing is trivial. In practice this rarely occurs — there is almost always a local model with lower quality but zero cost that is on the frontier for simple tasks. The frontier identifies where the genuine tradeoff exists.

Q: Why use ROUGE-1 for summarization instead of something like BERTScore?

A: ROUGE-1 is interpretable and deterministic — you can explain to a product manager why a summary scored 0.82 vs 0.74. BERTScore is more semantically accurate but adds a model dependency to the evaluation layer, creating a circular evaluation problem. The hybrid approach (ROUGE + semantic similarity) balances these concerns: ROUGE captures exact content overlap; semantic similarity catches paraphrased but correct outputs.

Q: What happens when a new model is released and pricing changes?

A: The quality baselines in config.py are seeded and reproducible. When pricing changes, re-running the profiler with updated cost configs triggers a new Pareto analysis. If a previously-dominated config becomes frontier (e.g., gpt-3.5-turbo price drops below gpt-4o-mini), the routing rules update automatically. The audit log captures which rule version was active at each decision point.

Q: How does complexity score work? Can it be gamed?

A: Complexity score is a deterministic function of prompt token count, vocabulary entropy (type-token ratio), and task-type baseline. It does not use an LLM for classification — that would be circular. The score is computed before any model call, is reproducible given the same input, and is logged in the audit record. Gaming it would require adversarially constructing prompts to inflate or deflate the score, which is a known limitation of any static complexity heuristic.

Q: How does this relate to the portfolio's failure mode thesis?

A: InferenceLens addresses failure mode #1: how does an AI system know it's working correctly? The answer here is: by having an independent verification signal — the Pareto frontier and PASS/WARN/FAIL verdict — that is computed from empirical profiling data, not from the model's own confidence score. The system is verified by evidence external to itself.

Q: What's the PASS/WARN/FAIL verdict based on?

A: PASS: all high-complexity tasks route to frontier models AND average quality across all tasks is above the floor (0.75). WARN: at least one dominated config is actively routing, or quality is between 0.65-0.75. FAIL: a critical task type routes to a dominated config, or average quality falls below 0.65. The verdict is generated per task type and aggregated globally.

7. Numbers to Know

Metric	Value	Context
Model tiers profiled	4	large, medium, small, local
Task types supported	3	summarization, classification, extraction
Quality floor	0.75	Below this → no routing recommendation
Cost savings minimum	20%	Must save >= 20% to justify routing down
Complexity bands	3	simple (<0.35), standard (0.35-0.70), complex (>0.70)
gpt-4o-mini vs gpt-4o	97% cheaper	0.88 vs 0.91 quality on summarization
local vs large	~100% cheaper	0.74 vs 0.91 quality — valid for simple tasks
Audit format	JSONL append-only	One record per inference call
Test coverage	48 tests passing	evaluator, pareto, router, integration

InferenceLens is part of a 13-repo Applied LLM Systems portfolio. All quality baselines use seeded synthetic data — reproducible without API calls. github.com/SidharthKriplani/inferencelens